

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 9

Professor Mario Pego

mario.pegó@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Criar arquivos Launch para
orquestrar e configurar
múltiplos nós ROS 2
simultaneamente

O que vamos ver

1

O problema de N terminais abertos

Por que orquestrar nós manualmente não escala

2

Anatomia de um arquivo Launch

LaunchDescription, Node() e generate_launch_description()

3

robo_hospitalar_launch.py

O launch file completo do robô hospitalar

4

Argumentos de launch

Configurando temperatura_limite direto na linha de comando

O que já construímos, nó por nó

Etapas 5, 6 e 7 — sempre um 'ros2 run' por vez

- controlador.py e estado_robo.py (Etapa 5): serviço e tópico, cada um seu terminal
- patrulha_server.py / patrulha_client.py e coordenador_vigilancia.py (Etapa 6): ação e reação a eventos
- monitor_transporte.py (Etapa 7): tópicos, serviço e um parâmetro configurável — temperatura_limite
- Hoje: um único arquivo que sobe vários desses nós de uma vez, com um único comando

O problema de escalar com 'ros2 run'

O que acontece quando o sistema tem mais de 1 ou 2 nós

- Cada 'ros2 run' ocupa um terminal — 3 nós significam 3 abas abertas, na ordem certa, sem esquecer nenhuma
- Reiniciar o sistema inteiro vira um processo manual: fechar tudo, lembrar os comandos, digitar de novo
- Passar um parâmetro diferente (como temperatura_limite) em cada terminal, de forma consistente, é fácil de errar
- Um arquivo de launch resolve os três problemas: descreve o sistema uma vez, em um arquivo versionável, e sobe tudo com 'ros2 launch'

Anatomia mínima de um launch file

Todo launch file em Python exporta uma função com este nome exato

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='meu_pacote',
            executable='meu_no',
        ),
    ])
```

`generate_launch_description()`

O ROS 2 procura exatamente essa função no arquivo. Ela devolve um `LaunchDescription` — uma lista de 'ações' (o mais comum é `Node`, mas existem outras, como veremos). Cada `Node()` aqui corresponde a um 'ros2 run' que você não vai mais digitar à mão.

Os argumentos mais usados de Node()

Praticamente todo launch file usa algum subconjunto destes

Argumento	Obrigatório?	Para que serve
package	Sim	Pacote onde está o executável
executable	Sim	Nome registrado em entry_points (setup.py)
name	Não	Renomeia o nó, sobrescrevendo o do __init__
output	Não	'screen' imprime os logs no terminal do launch
parameters	Não	Lista de dicts com valores de parâmetros
remappings	Não	Lista de tuplas (tópico antigo, tópico novo)

Configurando o launch pela linha de comando

DeclareLaunchArgument expõe um valor que muda sem editar o arquivo

```
from launch.actions import (
    DeclareLaunchArgument)
from launch.substitutions import (
    LaunchConfiguration)

meu_arg = DeclareLaunchArgument(
    'nome_do_argumento',
    default_value='valor_padrao',
)
# uso: LaunchConfiguration('nome_do_argumento')
```

Sempre string

O valor de um launch argument é sempre texto na linha de comando (ex: 'temperatura_limite:=25.0'). Quando ele alimenta um parâmetro numérico, o ROS 2 converte automaticamente — é o mesmo mecanismo do `--ros-args -p` da Etapa 7, só que descrito dentro do arquivo.

Onde o launch file fica no pacote

Um diretório novo, irmão de comunicacao_ros2/ e package.xml

```
comunicacao_ros2/  
├─ comunicacao_ros2/  
│   ├─ controlador.py  
│   ├─ estado_robo.py  
│   └─ monitor_transporte.py  
├─ launch/  
│   └─ robo_hospitalar_launch.py  
├─ package.xml  
└─ setup.py
```

launch/ não é o pacote Python

Repare que launch/ fica no mesmo nível de setup.py, não dentro da pasta comunicacao_ros2/ que tem os nós. É um diretório à parte — e por isso setup.py precisa ser avisado dele (próximo slide).

Criando o diretório launch/

Um comando simples, antes de escrever o arquivo

```
$ cd ~/tp1_ws/src/comunicacao_ros2  
$ mkdir launch  
$ touch launch/robo_hospitalar_launch.py
```

Nome do arquivo

A convenção é terminar em '_launch.py' — não é obrigatório para o Python, mas é o padrão que o resto da comunidade ROS 2 usa e que o setup.py do próximo slide já espera.

robo_hospitalar_launch.py — parte 1 de 4

Imports e o argumento de launch temperatura_limite

salvar em `comunicacao_ros2/launch/robo_hospitalar_launch.py`

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    limite_arg = DeclareLaunchArgument(
        'temperatura_limite',
        default_value='28.0',
        description='Limiar de alerta (°C)',
    )
```

robo_hospitalar_launch.py — parte 2 de 4

Os dois primeiros nós: controlador e estado_robo

salvar em `comunicacao_ros2/launch/robo_hospitalar_launch.py`

```
controlador_node = Node(  
    package='comunicacao_ros2',  
    executable='controlador',  
    name='controlador',  
    output='screen',  
)  
  
estado_robo_node = Node(  
    package='comunicacao_ros2',  
    executable='estado_robo',  
    name='estado_robo',  
    output='screen',  
)
```

robo_hospitalar_launch.py — parte 3 de 4

O terceiro nó, com o parâmetro alimentado pelo argumento de launch

salvar em `comunicacao_ros2/launch/robo_hospitalar_launch.py`

```
monitor_node = Node(  
    package='comunicacao_ros2',  
    executable='monitor_transporte',  
    name='monitor_transporte',  
    output='screen',  
    parameters=[  
        'temperatura_limite':  
            LaunchConfiguration('temperatura_limite'),  
    ],  
)
```

robo_hospitalar_launch.py — parte 4 de 4

O retorno: os quatro itens juntos no LaunchDescription

salvar em `comunicacao_ros2/launch/robo_hospitalar_launch.py`

```
return LaunchDescription([
    limite_arg,
    controlador_node,
    estado_robo_node,
    monitor_node,
])
```

Registrando launch/ no setup.py

Sem isso, o colcon build não copia o arquivo para o install/

salvar em `comunicacao_ros2/setup.py`

```
import os
from glob import glob
from setuptools import setup

package_name = 'comunicacao_ros2'

setup(
    # ... name, version, entry_points, etc.
    data_files=[
        # ... resource_index, package.xml
        (os.path.join('share', package_name, 'launch'),
         glob(os.path.join('launch', '*_launch.py'))),
    ],
)
```

`glob('*_launch.py')`

glob pega automaticamente qualquer arquivo em launch/ que termine em '_launch.py' — é por isso que o nome do arquivo importa. Adicione um novo launch file amanhã e ele entra sem editar essa linha de novo.

Compilando com o novo launch file

colcon build de novo — launch/ só existe no install/ depois disso

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash
```

Erro comum

'Package \'comunicacao_ros2\' not found' ao rodar ros2 launch quase sempre significa: esqueceu de rebuildar depois de criar launch/, ou esqueceu de adicionar a linha no setup.py.

Subindo o sistema inteiro com um comando

ros2 launch — três nós, um terminal só

```
$ ros2 launch comunicacao_ros2 \
  robo_hospitalar_launch.py

[INFO] [launch]: All log files can be found below...
[INFO] [controlador-1]: process started with pid [12345]
[INFO] [estado_robo-2]: process started with pid [12346]
[INFO] [monitor_transporte-3]: process started with pid [12347]
[controlador-1] [INFO] [controlador]: nó iniciado
[estado_robo-2] [INFO] [estado_robo]: nó iniciado
[monitor_transporte-3] [INFO] [monitor_transporte]: nó iniciado
```

O prefixo [nome-N]

ros2 launch numera cada processo na ordem do LaunchDescription e prefixa toda linha de log com esse nome — é assim que você distingue os três nós misturados no mesmo terminal.

Mudando o limiar sem editar o arquivo

Terminal 1: launch com argumento. Terminal 2: confirmando o valor

Terminal 1 — ros2 launch com argumento

```
$ ros2 launch comunicacao_ros2 \
  robo_hospitalar_launch.py \
  temperatura_limite:=25.0
```

Terminal 2 — ros2 param get

```
$ ros2 param get \
  /monitor_transporte \
  temperatura_limite

Double value is: 25.0
```

Mesma ideia da Etapa 7, lugar diferente

Antes, o valor entrava por --ros-args -p ou por 'ros2 param set' com o nó já rodando. Agora ele nasce certo, na hora de subir o sistema inteiro — sem tocar em nenhum ros2 run individual.

Remapeando tópicos no launch

Renomeia uma interface de um nó sem tocar no código-fonte dele

```
Node(  
  package='comunicacao_ros2',  
  executable='monitor_transporte',  
  name='monitor_transporte',  
  remappings=[  
    ('/temperatura_ambiente',  
     '/sensor/temperatura'),  
  ],  
)
```

Para que serve

Útil quando dois nós esperam nomes de tópico diferentes para a mesma informação, ou quando você quer rodar duas cópias do mesmo nó sem colisão. Fora do escopo do robô hospitalar de hoje, mas comum em sistemas maiores.

Reaproveitando outro launch file

IncludeLaunchDescription evita copiar e colar Node() entre arquivos

```
from launch.actions import (
    IncludeLaunchDescription)
from launch.launch_description_sources import (
    PythonLaunchDescriptionSource)

IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        'outro_pacote/launch/sensores_launch.py'
    )
)
```

Composição, não duplicação

Em sistemas com dezenas de nós, é comum ter um launch file por subsistema (sensores, navegação, braço) e um launch file 'principal' que só inclui os outros. Fora do escopo do AT, mas bom saber que existe.

Antes e depois do launch file

O mesmo sistema, duas formas de subir

- 3 terminais abertos, na ordem certa
- 3 comandos 'ros2 run' digitados à mão
- Parâmetro passado (ou esquecido) terminal por terminal
- Reiniciar tudo = repetir os 3 passos de novo

Depois: 1 comando

Um único 'ros2 launch comunicacao_ros2 robo_hospitalar_launch.py temperatura_limite:=25.0' sobe os três nós já com o limiar certo. O arquivo fica versionado no Git e descreve o sistema inteiro — qualquer colega roda exatamente o mesmo grafo de nós, com os mesmos parâmetros, sem decorar comando nenhum.

ros2 run vs. ros2 launch

Não são concorrentes — launch é construído sobre run

Situação	Quando usar
Testar um único nó isolado, depurando	ros2 run
Subir o sistema completo (2+ nós)	ros2 launch
Precisa de parâmetros/remappings consistentes	ros2 launch
Entregar um comando único para outra pessoa rodar	ros2 launch

Launch files sem armadilhas comuns

- Editou launch/ ou setup.py? Rode 'colcon build' de novo — assim como um .py de nó, o launch file só existe no install/ depois do build
- Um argumento de launch com nome errado na linha de comando falha na hora — 'ros2 launch --show-args <pacote> <arquivo>' lista os nomes válidos
- output='screen' é opcional, mas sem ele os logs do nó só vão para arquivo — em desenvolvimento, quase sempre vale a pena usar
- Dois Node() com o mesmo 'executable' e sem 'name'/'namespace' diferentes colidem no grafo — dê nomes explícitos quando subir mais de uma instância

OK

`launch/ + registro no setup.py`

Onde salvar um launch file e empacotá-lo corretamente

OK

`robo_hospitalar_launch.py`
completo

Três nós sobem com um único 'ros2 launch'

OK

`DeclareLaunchArgument +`
`LaunchConfiguration`

Expor e sobrescrever um valor pela linha de comando

OK

`ros2 run vs ros2 launch`

Explicar quando cada um faz sentido

Próximo passo

Com o launch file orquestrando controlador, estado_robo e monitor_transporte, falta a peça que dá nome à Competência 4 do AT: colocar o robô hospitalar para existir fisicamente num simulador. Na próxima competência, o mesmo URDF da Etapa 8 sobe dentro do Gazebo, com o robot_state_publisher publicando as transformadas e o monitor_transporte operando sobre o robô simulado — tudo orquestrado por um único launch file, igual ao que acabamos de construir.

Simular sistemas robóticos com Gazebo no ecossistema ROS 2

O que vamos ver

1

Por que simular antes do robô real

O papel do Gazebo no ciclo de desenvolvimento

2

robot_state_publisher e spawn_entity.py

Como o URDF vira um robô de verdade dentro do simulador

3

hospital_gazebo.launch.py

O launch file que sobe o mundo, o robô e o monitoramento juntos

4

Validação

ros2 node list, ros2 topic list e ros2 param get com o robô simulado

As três peças que já temos prontas

Etapas 7, 8 e a Competência 1 desta etapa

- `monitor_transporte.py` (Etapa 7): já publica, expõe serviço e aceita `temperatura_limite` como parâmetro
- `robo_hospitalar.urdf` (Etapa 8): descreve `base_link`, `roda_esq`, `roda_dir` e `roda_apoio`, já validado com `check_urdf`
- `robo_hospitalar_launch.py` (Competência 1): já sabemos orquestrar múltiplos nós com argumentos configuráveis
- Hoje: um launch file novo que soma um quarto elemento — o próprio simulador — aos três de cima

Por que simular antes do robô físico

O papel do Gazebo no ciclo de desenvolvimento

- Até aqui, o robo_hospitalar existia só como texto: um arquivo URDF e nós que trocam mensagens, sem nada 'de verdade' se movendo
- O Gazebo é um simulador de física: aplica gravidade, atrito e colisão ao mesmo URDF que descrevemos na Etapa 8, dando ao robô um corpo que ocupa espaço e reage a forças
- Testar controle, sensores e lógica de decisão em simulação evita colidir um robô real de R\$ dezenas de milhares contra uma parede por um bug de código
- O mesmo grafo de nós (tópicos, serviços, parâmetros) que já construímos funciona sem mudança nenhuma — só passa a operar sobre um robô simulado, não mais um objeto totalmente abstrato

Instalando o Gazebo Classic para o ROS 2 Humble

Um único pacote apt integra os dois mundos

```
$ sudo apt install ros-humble-gazebo-ros-pkgs
```

O papel do gazebo_ros

gazebo_ros é a ponte entre os dois mundos: expõe o Gazebo (um simulador C++ independente) como pacotes ROS 2 — é de lá que vêm o launch file gazebo.launch.py e o executável spawn_entity.py que vamos usar a seguir.

O papel do robot_state_publisher

Ele lê o CONTEÚDO do URDF, não só o caminho do arquivo

```
with open('robo_hospitalar.urdf') as f:
    robot_description = f.read()

Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    parameters=[{
        'robot_description': robot_description,
    }],
)
```

Por que o texto inteiro

O parâmetro 'robot_description' espera a STRING do XML, não um caminho de arquivo. O robot_state_publisher lê essa string, calcula a posição de cada link a partir dos joints, e publica tudo em /tf — é o que RViz e Gazebo usam para saber onde cada peça do robô está.

spawn_entity.py: inserindo o robô no mundo

Lê o URDF do tópico /robot_description e cria o modelo dentro do Gazebo

```
Node(  
    package='gazebo_ros',  
    executable='spawn_entity.py',  
    arguments=[  
        '-topic', 'robot_description',  
        '-entity', 'robo_hospitalar',  
    ],  
)
```

-topic vs -file

'-topic robot_description' reaproveita o mesmo texto que o robot_state_publisher já publicou — por isso ele precisa subir primeiro. A alternativa '-file caminho.urdf' releria o arquivo direto, ignorando o que já está no grafo. '-entity' só dá um nome ao modelo dentro do Gazebo.

ExecuteProcess: rodando qualquer comando no launch

Uma quinta 'ação' de launch, ao lado de Node e IncludeLaunchDescription

```
from launch.actions import ExecuteProcess

ExecuteProcess(
    cmd=['ros2', 'topic', 'echo', '/algum_topico'],
    output='screen',
)
```

Não precisa ser um nó ROS 2

Node() só serve para executáveis registrados em entry_points de um pacote ROS 2.

ExecuteProcess roda QUALQUER comando de terminal — aqui, um 'ros2 topic echo' comum, para deixar os alertas visíveis assim que o sistema sobe, sem precisar de um terminal extra.

Estrutura

```
comunicacao_ros2/  
├─ comunicacao_ros2/  
│   └─ ... (nós Python)  
├─ launch/  
│   ├── robo_hospitalar_launch.py  
│   └─ hospital_gazebo_launch.py  
├─ urdf/  
│   └─ robo_hospitalar.urdf  
├─ package.xml  
└─ setup.py
```

Registrando urdf/ no setup.py

A mesma linha de padrão de launch/, com outra extensão de arquivo

salvar em `comunicacao_ros2/setup.py`

```
setup(  
    # ... name, version, entry_points, etc.  
    data_files=[  
        # ... resource_index, package.xml  
        (os.path.join('share', package_name, 'launch'),  
         glob(os.path.join('launch', '*_launch.py'))),  
        (os.path.join('share', package_name, 'urdf'),  
         glob(os.path.join('urdf', '*.urdf'))),  
    ],  
)
```

Uma tupla por diretório

Cada diretório extra do pacote (launch/, urdf/, e no futuro talvez config/ ou meshes/) ganha sua própria tupla em data_files, com seu próprio glob. É o mesmo padrão se repetindo — não um mecanismo novo.

hospital_gazebo_launch.py — parte 1 de 6

Imports e os caminhos dos dois pacotes envolvidos

salvar em `comunicacao_ros2/launch/hospital_gazebo_launch.py`

```
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, ExecuteProcess
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node

def generate_launch_description():
    pkg_gazebo_ros = get_package_share_directory('gazebo_ros')
    pkg_comunicacao = get_package_share_directory('comunicacao_ros2')
```

hospital_gazebo_launch.py — parte 2 de 6

Incluindo o Gazebo e lendo o conteúdo do URDF

salvar em `comunicacao_ros2/launch/hospital_gazebo_launch.py`

```
gazebo = IncludeLaunchDescription(  
    PythonLaunchDescriptionSource(  
        os.path.join(  
            pkg_gazebo_ros, 'launch', 'gazebo.launch.py')  
        )  
    )  
)  
  
urdf_path = os.path.join(  
    pkg_comunicacao, 'urdf', 'robo_hospitalar.urdf')  
with open(urdf_path, 'r') as f:  
    robot_description = f.read()
```

hospital_gazebo_launch.py — parte 3 de 6

O robot_state_publisher, com o texto do URDF como parâmetro

salvar em `comunicacao_ros2/launch/hospital_gazebo_launch.py`

```
robot_state_publisher = Node(  
    package='robot_state_publisher',  
    executable='robot_state_publisher',  
    output='screen',  
    parameters=[{  
        'robot_description': robot_description,  
    }],  
)
```

hospital_gazebo_launch.py — parte 4 de 6

spawn_entity.py: cria o modelo dentro do Gazebo

salvar em `comunicacao_ros2/launch/hospital_gazebo_launch.py`

```
spawn_entity = Node(  
    package='gazebo_ros',  
    executable='spawn_entity.py',  
    arguments=[  
        '-topic', 'robot_description',  
        '-entity', 'robo_hospitalar',  
    ],  
    output='screen',  
)
```


hospital_gazebo_launch.py — parte 5 de 6

O monitor_transporte, já com temperatura_limite fixado em 27.0

salvar em `comunicacao_ros2/launch/hospital_gazebo_launch.py`

```
monitor_node = Node(  
    package='comunicacao_ros2',  
    executable='monitor_transporte',  
    name='monitor_transporte',  
    output='screen',  
    parameters=[{'temperatura_limite': 27.0}],  
)  
  
echo_alertas = ExecuteProcess(  
    cmd=['ros2', 'topic', 'echo', '/alertas', 'std_msgs/msg/String'],  
    output='screen',  
)
```

hospital_gazebo_launch.py — parte 6 de 6

O retorno: os cinco itens juntos, na ordem em que devem subir

salvar em `comunicacao_ros2/launch/hospital_gazebo_launch.py`

```
return LaunchDescription([
    gazebo,
    robot_state_publisher,
    spawn_entity,
    monitor_node,
    echo_alertas,
])
```

Compilando com urdf/ e o novo launch file

colcon build de novo — dois diretórios novos desde a última vez

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash
```

Sem GUI no ambiente?

Se sua VM não tem aceleração gráfica, o Gazebo completo (gzserver + gzclient) pode travar ao abrir. Nesse caso, adicione 'gui:=false' ao comando do próximo slide para rodar em modo headless (só o gzserver, sem janela).

Subindo o mundo, o robô e o monitoramento juntos

ros2 launch — Gazebo abre, o robô aparece, os alertas já aparecem no mesmo terminal

```
$ ros2 launch comunicacao_ros2 \  
  hospital_gazebo_launch.py  
  
# ambiente sem GUI:  
$ ros2 launch comunicacao_ros2 \  
  hospital_gazebo_launch.py gui:=false  
  
[INFO] [spawn_entity.py-2]: process started...  
[INFO] [monitor_transporte-3]: nó iniciado
```

A ordem importa na prática

Mesmo com os cinco itens listados juntos no return, o Gazebo leva alguns segundos para abrir antes do spawn_entity conseguir inserir o robô — é normal ver uma pequena pausa nos logs antes do modelo aparecer.

Confirmando o sistema simulado no ar

Terminal 1: nós ativos. Terminal 2: tópicos, incluindo /tf

Terminal 1 — ros2 node list

```
$ ros2 node list
/gazebo
/robot_state_publisher
/monitor_transporte
```

Terminal 2 — ros2 topic list

```
$ ros2 topic list
/tf
/tf_static
/alertas
/temperatura_ambiente
```

/tf é a prova do robot_state_publisher

Se /tf aparece publicando, o robot_state_publisher está calculando a posição de cada link a partir do URDF — é a evidência de que a descrição da Etapa 8 está sendo usada de verdade, não só validada com check_urdf.

Confirmando o parâmetro no robô já simulado

ros2 param get — critério de validação do Exercício 4

```
$ ros2 param get /monitor_transporte \
  temperatura_limite
```

```
Double value is: 27.0
```

Veio do launch file, não de um ros2 param set

27.0 foi fixado direto em parameters=[{'temperatura_limite': 27.0}] na parte 5 de 6 — mostra que um launch file pode configurar um nó tão bem quanto um --ros-args -p ou um param set manual.

Problemas comuns ao integrar com o Gazebo

- 'Unable to connect to gzserver' — o Gazebo demorou para subir; rode o launch de novo, ou aumente a pausa antes do `spawn_entity` em sistemas mais lentos
- `spawn_entity` falha com 'entity already exists' — um Gazebo de uma execução anterior ainda está aberto; feche todos os processos `gzserver/gzclient` antes de relançar
- Robô aparece no Gazebo mas `/tf` não publica — quase sempre falta o `robot_state_publisher` no `LaunchDescription`, ou o parâmetro `robot_description` está vazio
- `gui:=false` não é mágica — ele evita a janela 3D, mas a física ainda roda; é a forma correta de validar em CI/VM sem tela, não um 'modo mais rápido'

OK

`gazebo_ros` instalado

Ponte entre o Gazebo e o ROS 2

OK

`robot_state_publisher +
spawn_entity.py`

URDF vira um robô dentro do simulador

OK

`hospital_gazebo_launch.py`
completo

mundo, robô e monitoramento sobem juntos com um comando

OK

`ros2 node list / topic list /
param get`

Validação de que o sistema simulado está de fato no ar

Fim do conteúdo do DR1

Com o robô simulado no Gazebo, publicando `/tf` e rodando o `monitor_transporte` por cima, as quatro competências do AT estão completas: arquitetura e mecanismos de comunicação (Exercícios 1 e 2), parâmetros e URDF (Exercício 3) e launch file com simulação física (Exercício 4). O próximo passo é ler o enunciado completo do Assessment e montar o relatório final, reunindo prints e justificativas de tudo que foi construído desde a Etapa 3.